KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Ayurveda-and-Home-Remedies-Knowledge-Bot

Project ID: PRJ-076

Team ID: T076

Team GitHub Repository: https://github.com/Balu-Balaji031/Ayurveda-and-Home-Remedies-Knowledge-Bot

Student Name	Registration Number	Individual GitHub Link
P. Balaji	99230040050	https://github.com/Balu-Balaji031
K. R. Hemadri	99230040033	https://github.com/hemadrikr6-cell/GEN-AI.git
S. Venumadhav Redddy	99230041160	https://github.com/venumadhavreddy6/Gen-AI.git
S. Sumanth	99230040058	https://github.com/sumanth1831/GenAI.git
T. Nikhil Sai Kumar	99230040439	https://github.com/nikhilsai2/GEN--AI.git

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction ..... Section 1
2. Problem Statement & Objectives ..... Section 2
3. System Architecture (RAG Pipeline) ..... Section 3
4. Implementation Details & Tech Stack ..... Section 4
5. Results and Testing ..... Section 5
6. Conclusion & Future Scope ..... Section 6

1 Introduction

Ayurveda and traditional home-remedy information is available across books, manuals, and other reference documents. However, users may find it difficult to locate information about a particular health concern, ingredients, preparation methods, and precautions in lengthy PDF files. A simple keyword search may also miss relevant passages when the wording of a question differs from the wording used in a document.

AyurBot is an Ayurveda and Home Remedies Knowledge Bot that provides a conversational way to explore information from a collection of PDF references. It uses Retrieval-Augmented Generation (RAG) to search for relevant passages and provide them as context to a language model before a response is generated. The application also supports remedy recommendations, selected-remedy explanations, and follow-up questions.

AyurBot is intended for general information and educational use. It is not a medical diagnostic system and should not replace advice, diagnosis, or treatment from a qualified healthcare professional.

2 Problem Statement & Objectives

Problem: Information about Ayurveda and home remedies is spread across multiple documents. Users may spend considerable time searching these files and may have difficulty identifying relevant precautions or deciding when a home remedy is not appropriate. General-purpose AI responses may also include information that is not supported by the available reference documents.

Objectives:

- To develop a chatbot that allows users to ask questions about Ayurveda and home remedies.
- To retrieve relevant passages from the available PDF knowledge base using semantic search.
- To generate responses using retrieved document context and provide source details where available.
- To present remedy recommendations and provide further details when a user selects a remedy.
- To support follow-up questions while retaining relevant conversation context.
- To provide user authentication and per-user chat history using Firebase services.
- To include precautions and appropriate guidance to seek professional medical help.

3 System Architecture (RAG Pipeline)

The AyurBot application follows two main stages: knowledge preparation and query processing. The frontend sends user questions to the backend. The backend retrieves relevant information from ChromaDB and uses the retrieved passages as context for the configured language model.

3.1 Data Ingestion Phase

The knowledge base is prepared from Ayurveda and home-remedy PDF documents. The backend extracts text from the PDFs, divides the extracted content into smaller chunks, converts the chunks into vector embeddings, and stores the embeddings with document metadata in ChromaDB.

1. PDF documents are read and their text is extracted using PyMuPDF.
2. The extracted text is divided into smaller passages suitable for retrieval.
3. The sentence-transformers/all-MiniLM-L6-v2 model converts the passages into embeddings.
4. The embeddings and associated metadata are stored in the ChromaDB collection ayurbot_knowledge_base.

The development knowledge-base snapshot recorded 361 chunks in ChromaDB. This count describes the tested development state and may change when documents are updated.

3.2 Query & Generation Phase

When a user submits a question, the backend converts the question into an embedding using the same embedding model. ChromaDB performs a similarity search and returns relevant passages from the stored PDF content. The retrieved passages, user question, conversation context, and safety instructions are then supplied to the language model to generate a response.

Google Gemini is configured as the primary language-model provider. Groq is configured as a fallback provider for eligible temporary failures of the primary provider. The backend validates the generated response and returns it to the frontend together with available source information.

3.3 Remedy Recommendation and Follow-up

For a remedy request, the system retrieves relevant passages and presents remedy options when the knowledge base supports them. After the user selects an option, the backend can generate a more detailed explanation covering the remedy, ingredients, preparation, possible usefulness, precautions, and situations in which professional advice should be obtained. Follow-up questions use the selected remedy and relevant conversation context.

When the reference PDFs do not contain relevant information, any general AI-generated suggestions must be identified separately from PDF-based information. The system should not present unsupported suggestions as verified source material.

RAG Processing Flow

User question → React / TypeScript frontend → FastAPI backend → Query embedding → ChromaDB retrieval → Retrieved PDF context → Gemini (primary) / Groq (fallback) → Response and source details → Chat interface

4 Implementation Details & Tech Stack

Component	Technology Used	Purpose
Frontend	React, TypeScript, Vite, Tailwind CSS	Building the chatbot interface and handling user interaction.
Backend	Python, FastAPI	Handling API requests and application logic.
RAG orchestration	LangChain	Connecting document processing, retrieval, and language-model workflows.
Vector Database	ChromaDB	Storing and retrieving document embeddings.
Embedding Model	Hugging Face sentence-transformers/all-MiniLM-L6-v2	Converting text passages and questions into vector representations.
PDF Processing	PyMuPDF	Extracting text from PDF reference documents.
Primary LLM	Google Gemini	Generating responses using retrieved context.
Fallback LLM	Groq API (openai/gpt-oss-20b)	Providing an alternative response-generation path for eligible provider failures.
Authentication	Firebase Authentication	Managing user registration and sign-in.
Database	Cloud Firestore	Maintaining user profile and chat-history data.

4.1 Main Application Features

- User registration and login.
- Conversational question-and-answer interface.
- PDF-based remedy recommendations and source information where available.
- Detailed explanation of a selected remedy.
- Context-aware follow-up questions.
- Per-user conversation history.
- Primary Gemini provider with Groq fallback handling.

4.2 Safety Considerations

AyurBot provides information for educational purposes only. It should not diagnose a condition, promise a cure, or recommend stopping prescribed treatment. The system should avoid inventing dosages, treatment durations, and source references. Users should consider allergies, pregnancy, age, existing health conditions, and possible medicine interactions. Severe, persistent, or worsening symptoms require advice from a qualified healthcare professional.

5 Results and Testing

The project was checked through backend tests and live API requests. The recorded development test run covered provider handling, remedy workflows, and chat API behavior. The outcomes below describe software tests and are not a measure of clinical accuracy or medical effectiveness.

Test Area	Test Performed	Recorded Result
Provider manager	Provider selection and fallback handling.	10 tests passed.
Remedy workflow	Recommendation and selected-remedy explanation.	12 tests passed.
Chat API	Chat and follow-up endpoint behavior.	12 tests passed.
Recommendation endpoint	Live recommendation request.	HTTP 200 observed.
Remedy explanation	Live explanation and source handling.	HTTP 200 observed.
Follow-up endpoint	Follow-up and context retention.	HTTP 200 observed.
Fallback scenario	Simulated eligible Gemini failure with Groq fallback.	Successful fallback responses observed.
ChromaDB	Check of existing development collection.	361 chunks retained in recorded snapshot.

These tests confirm the behavior of the tested software workflows. They do not establish medical efficacy, clinical safety, diagnostic accuracy, or complete coverage of all diseases and remedies. Further evaluation with a larger, independently reviewed test set is required.

6 Conclusion & Future Scope

6.1 Conclusion

AyurBot provides a conversational interface for exploring Ayurveda and home-remedy information contained in a collection of PDF documents. By combining semantic retrieval through ChromaDB with language-model response generation, the application helps users find relevant reference passages without manually searching every document. The project also includes user authentication, chat history, remedy selection, detailed explanations, and contextual follow-up.

The implementation uses Gemini as the primary language model and Groq as a fallback for eligible provider failures. The application is designed to make reference information easier to access while retaining important limitations: generated answers depend on the quality of the source documents and should not be treated as a substitute for professional medical advice.

6.2 Future Scope

- Add and maintain a larger collection of reliable, reviewed Ayurveda and public-health reference documents.
- Improve multilingual interaction for English, Tamil, and Telugu.
- Provide clearer source details, including document names and page references where available.
- Introduce expert review and user feedback to identify incomplete or potentially unsafe responses.
- Evaluate retrieval quality and response faithfulness using a larger test dataset.
- Improve deployment and persistent storage arrangements for the vector database.
- Explore voice interaction and image-based document understanding.